The beauty of this pattern lies in the fact that an arbitrary number of decorators can be used to wrap the main component, which allows you to fine-tune the functionality of the component without resorting to adding subclasses for each permutation. The component and the decorator share a common interface; the decorator is constructed with an instance of that interface, and this allows decorators to wrap other decorators. Decorators delegate their implementations of the interface methods to their wrapped objects while adding their own additions. First, let's look at a class diagram that shows how the component, interface, and decorator(s) are related:

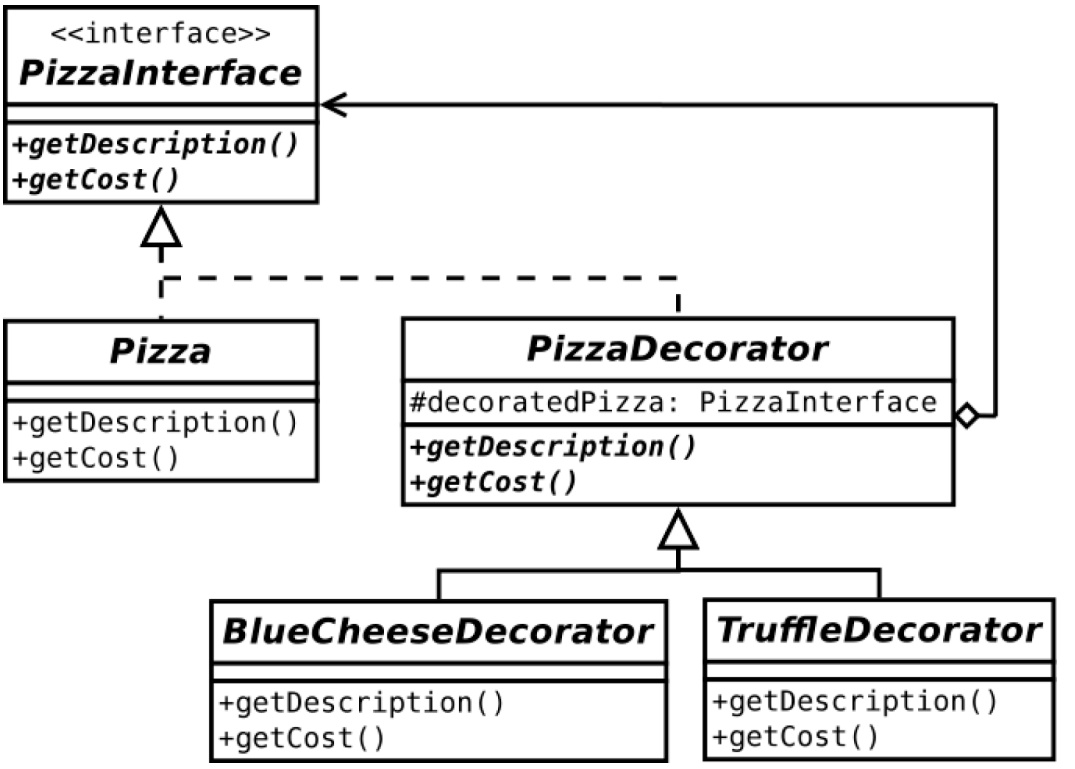


Figure 10-20. *Decorator pattern*

The following code shows simple implementations for a `Pizza` component class, an abstract base decorator (which really only exists here to save having to define the same constructor in each subclass), and two concrete decorator classes:

```
// PizzaInterface defines two public methods: getCost() and
getDescription()
```